```
# Plant Leaf Disease Classification with a CNN

## Objective
Classify images into one of 5 plant-health categories using a Convolutional
Neural Network (CNN) built with TensorFlow/Keras. This is a multi-class image
classification problem.

## Dataset
- **Size:** 60 images (224×224 RGB) in `Images/train`, split 48 training / 12 validation
- **Classes (5):** Blight, Healthy, Leaf_spot, Mildew, Rust
- **Note:** [Add the source here, or state that these are synthetic/placeholder images.]

## Methodology
1. Loaded images with `image_dataset_from_directory` (batch size 32, seed 42,
   20% validation split)
2. Visualized sample images with their labels
3. Built a Sequential CNN:
   - Rescaling (1/255 normalization)
   - Conv2D(32) → MaxPooling → Conv2D(64) → MaxPooling
   - Flatten → Dense(64, ReLU) → Dense(5, Softmax)
   - About 11.96M trainable parameters
4. Compiled with Adam and sparse categorical cross-entropy
5. Trained for 10 epochs
6. Plotted accuracy/loss curves, a classification report, a confusion matrix,
   and sample predictions

## Results
- Training accuracy after 10 epochs: **~60%**
- Validation accuracy: **75%** (loss 1.19)

## Limitations and Future Work
- The validation split was not stratified, so the 12 validation images are all
  from the Rust class. The 75% validation accuracy therefore only reflects Rust
  and says little about the other four classes.
- The dataset is very small (12 images per class), and the model is large for
  this much data and is likely to overfit.
- Next steps: add more images, use a stratified split, apply data augmentation,
  and try transfer learning (e.g. MobileNetV2 or ResNet).

## Tech Stack
Python, TensorFlow/Keras, NumPy, Matplotlib, Scikit-learn
```

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "Images/train",
    image_size = (224, 224),
    batch_size = 32,
    seed = 42
)
```

```
Found 60 files belonging to 5 classes.
```

```
print(train_ds.cardinality().numpy())   # number of batches
```

```
2
```

```
count = sum(len(x) for x, y in train_ds)
print(count)   # should print 15
```

```
60
```

```
class_names = train_ds.class_names
print(class_names)
```

```
['Blight', 'Healthy', 'Leaf_Spot', 'Mildew', 'Rust']
```

```
import tensorflow as tf
import os
```

```
# Image settings
img_size = (224, 224)
batch = 32
data_dir = "/content/Images"

# Load training images and split them into training and validation sets
train_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(data_dir, "train"),
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=img_size,
    batch_size=batch
)

# Load validation images from the same training folder
val_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(data_dir, "train"),
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=img_size,
    batch_size=batch,
    shuffle=False
)

# Get and display class names
class_names = train_ds.class_names
print("Class names:", class_names)

# Improve data-loading performance
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)

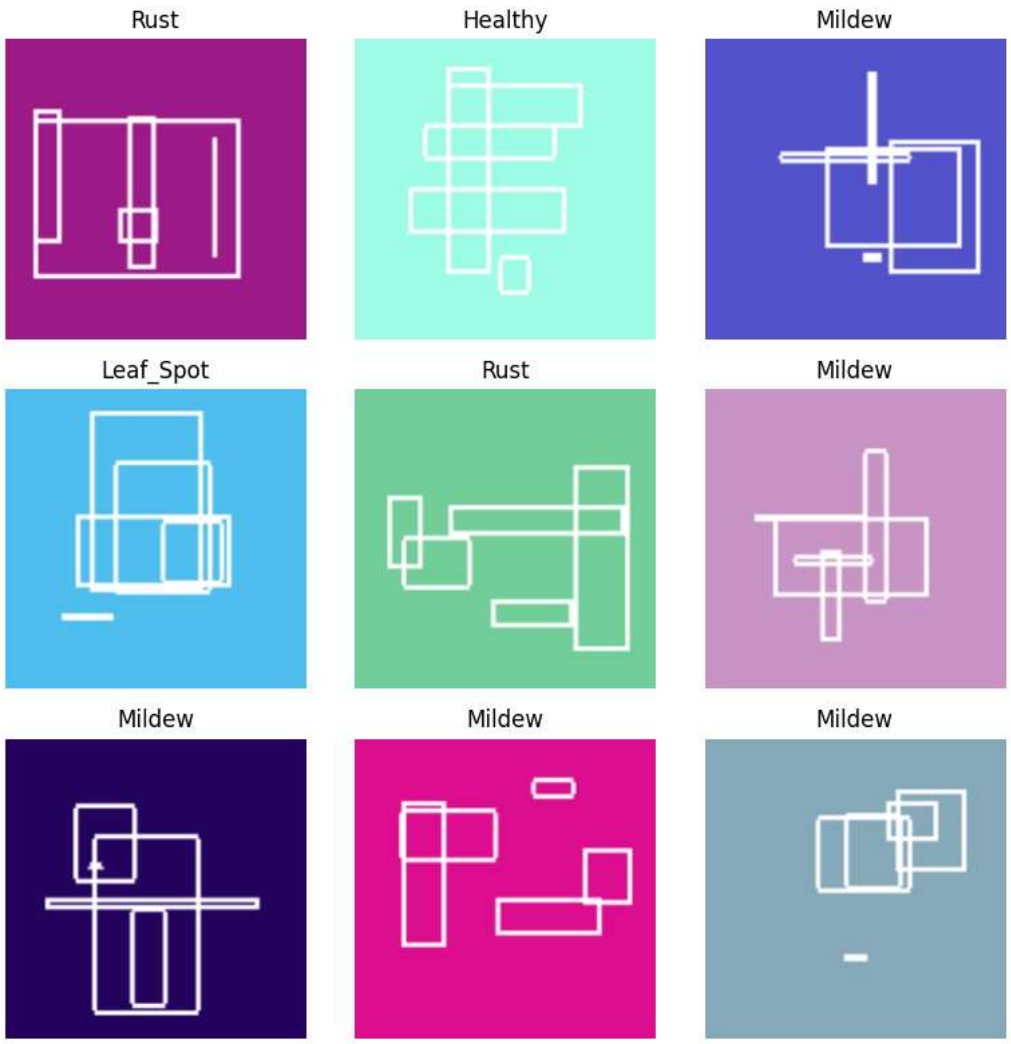
print("Training and validation datasets loaded successfully!")
```

```
Found 60 files belonging to 5 classes.
Using 48 files for training.
Found 60 files belonging to 5 classes.
Using 12 files for validation.
Class names: ['Blight', 'Healthy', 'Leaf_Spot', 'Mildew', 'Rust']
Training and validation datasets loaded successfully!
```

```
plt.figure(figsize=(8, 8))

for images, labels in train_ds.take(1):
    for i in range(min(9, len(images))):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.tight_layout()
plt.savefig("/content/plant_images.png", dpi=300, bbox_inches="tight")
plt.show()
```




```
Cnn = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./255),               # normalization

    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(len(class_names), activation="softmax")
])

Cnn.compile(optimizer="adam",
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"])
```

```
history = Cnn.fit(train_ds, validation_data=val_ds, epochs=10)
```

```
Epoch 1/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 3s/step - accuracy: 0.1875 - loss: 6.7145 - val_accuracy: 0.5000 - val_loss: 1.4418
Epoch 2/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - accuracy: 0.2292 - loss: 7.8459 - val_accuracy: 0.2500 - val_loss: 3.2663
Epoch 3/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 11s 6s/step - accuracy: 0.3333 - loss: 4.3242 - val_accuracy: 0.2500 - val_loss: 2.1438
Epoch 4/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 2s/step - accuracy: 0.3542 - loss: 2.9740 - val_accuracy: 0.2500 - val_loss: 1.4484
Epoch 5/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.2917 - loss: 1.9440 - val_accuracy: 0.3333 - val_loss: 1.5143
Epoch 6/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 2s/step - accuracy: 0.3958 - loss: 1.3062 - val_accuracy: 0.5000 - val_loss: 1.7895
Epoch 7/10
```

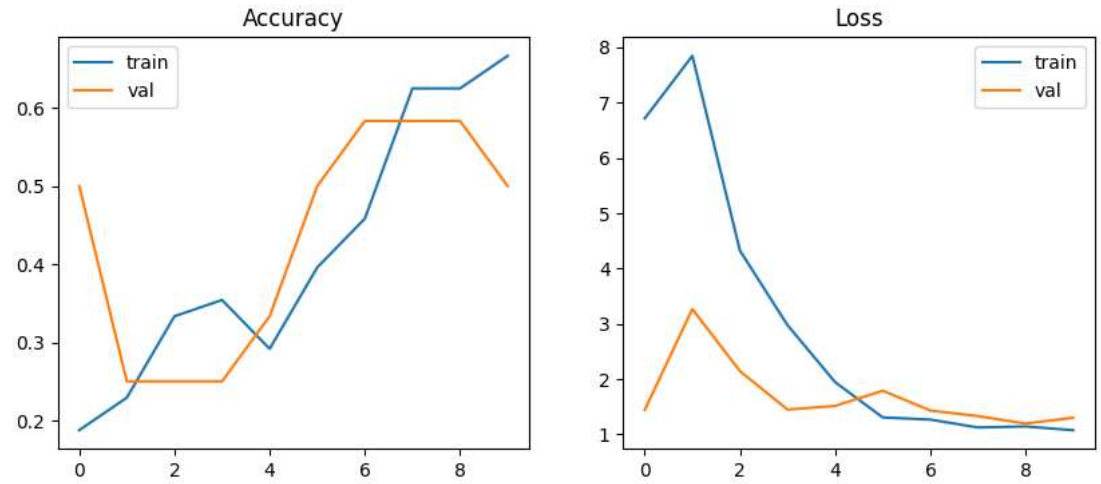
```
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.4583 - loss: 1.2670 - val_accuracy: 0.5833 - val_loss: 1.4314
Epoch 8/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - accuracy: 0.6250 - loss: 1.1246 - val_accuracy: 0.5833 - val_loss: 1.3301
Epoch 9/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 2s/step - accuracy: 0.6250 - loss: 1.1430 - val_accuracy: 0.5833 - val_loss: 1.1953
Epoch 10/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.6667 - loss: 1.0753 - val_accuracy: 0.5000 - val_loss: 1.3003
```

```
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="val")
plt.title("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="train")
plt.plot(history.history["val_loss"], label="val")
plt.title("Loss")
plt.legend()

plt.show()
```




```
Cnn.evaluate(val_ds)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 274ms/step - accuracy: 0.5000 - loss: 1.3003
[1.3002638816833496, 0.5]
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
from sklearn.metrics import classification_report, confusion_matrix

y_true = np.concatenate([y.numpy() for x, y in val_ds])
y_pred = Cnn.predict(val_ds)
y_pred_classes = np.argmax(y_pred, axis=1)

print(classification_report(
    y_true, y_pred_classes,
    labels=range(len(class_names)),
    target_names=class_names,
    zero_division=0))

print(confusion_matrix(y_true, y_pred_classes, labels=range(len(class_names))))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 327ms/step
              precision    recall  f1-score   support

      Blight       0.00      0.00      0.00         0
     Healthy       0.00      0.00      0.00         0
   Leaf_Spot       0.00      0.00      0.00         0
      Mildew       0.00      0.00      0.00         0
        Rust       1.00      0.50      0.67        12

    accuracy                           0.50        12
   macro avg       0.20      0.10      0.13        12
```

```
weighted avg       1.00      0.50      0.67        12

[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [3 0 1 2 6]]
```

```
for images, labels in val_ds.take(1):
    preds = Cnn.predict(images)
    for i in range(min(3, len(images))):
        plt.figure(figsize=(3, 3))
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(f"Actual: {class_names[labels[i]]}\nPredicted: {class_names[np.argmax(preds[i])]}")
        plt.axis("off")
        plt.tight_layout()
        plt.savefig(f"/content/plant_image_{i+1}.png", dpi=300, bbox_inches="tight")
        plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 382ms/step
```

